

Computer Science Extended Essay:

Empirical runtime comparison of Prim's and Kruskal's algorithms for minimum spanning trees under varying graph density

Research question:

For randomly generated weighted graphs with 2000 nodes, how does increasing graph density affect the measured runtime of Kruskal's algorithm with Disjoint Set Union (path compression and union by size optimizations) and Prim's algorithm implemented with a min-heap priority-queue?

Words: 3492

Candidate code: lzd185

1. Introduction.....	4
2. Graph Theory.....	5
2.1 Graph representation	7
3. Kruskal's Algorithm.....	9
3.1 Disjoint sets data structure.....	9
3.2 Implementing Kruskal's algorithm.....	13
4. Prim's Algorithm	14
4.1 Min-heap data structure.....	14
4.2 Implementing Prim's algorithm.....	16
5. Hypothesis and Applied Theory	18
6. Methodology	20
6.1 Independent variable	20
6.2 Dependent variable.....	20
6.3 Controlled variables	20
6.4 Procedure.....	21
7. Processing data.....	23
8. Results evaluation and discussion.....	26
8.1 Summary of results	26
8.2 Interpretation and evaluation of results	26
8.3 Further evaluation of limitations.....	27
9. Conclusion	27
Bibliography	28
Appendices.....	30
Appendix A: Algorithm implementations.....	30
A1: Disjoint Set Union (Laaksonen).....	30
A2: Kruskal's algorithm.....	30
A3: Prim's algorithm	31
Appendix B: Full code used for my experiment	32
Appendix C: Written permission from Antti Laaksonen to use his DSU implementation	36
C1: Email I sent asking for permission	36

C2: His response36

1. Introduction

Research question:

For randomly generated weighted graphs with 2000 nodes, how does increasing graph density affect the measured runtime of Kruskal's algorithm with Disjoint Set Union (path compression and union by size optimizations) and Prim's algorithm implemented with a min-heap priority-queue?

This essay will investigate the runtimes of Kruskal's and Prim's minimum spanning tree (MST) algorithms. The theoretical time complexity of both algorithms will be explored and compared to experimental results. Minimum spanning trees appear in many real-life computational problems containing graphs such as network design, taxonomy, cluster analysis, tree construction, image recognition and segmentation, and circuit design [1]. Graphs in each application have varying densities, therefore it is important to have an understanding of which algorithm is efficient under different graph densities.

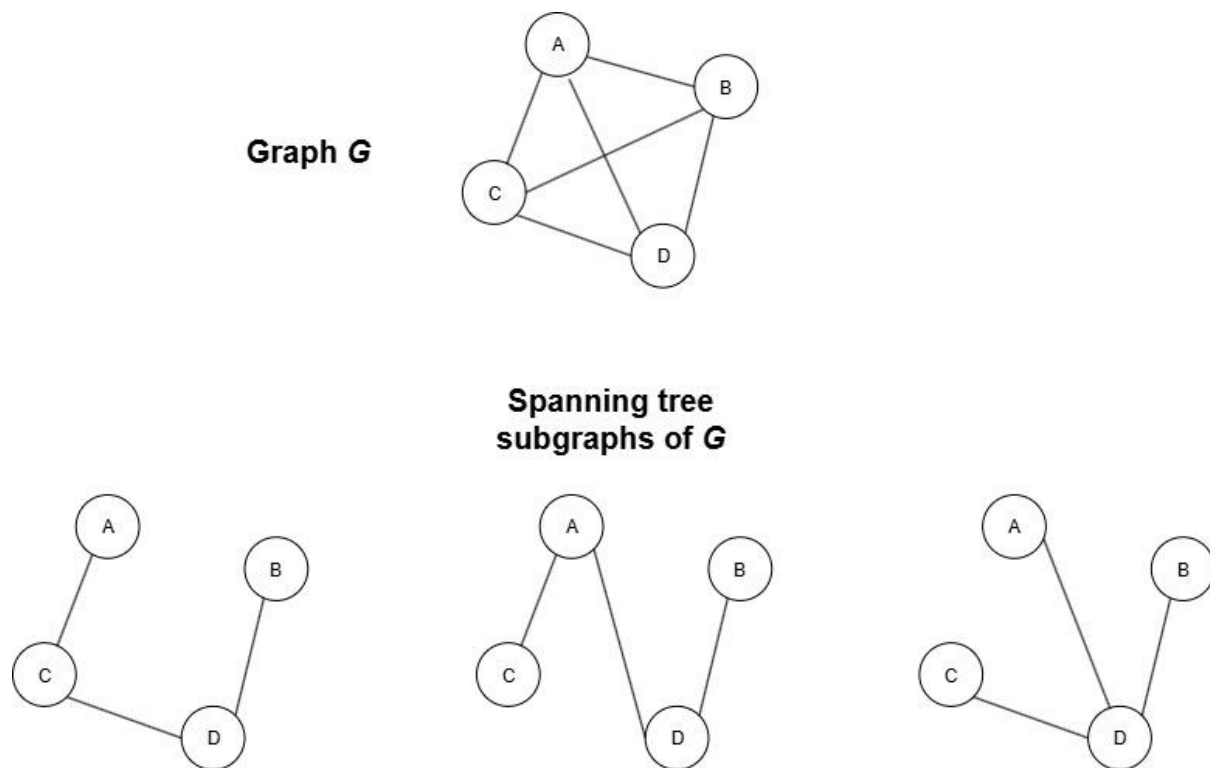
2. Graph Theory

A graph consists of nodes and edges, where each edge connects a pair of nodes. In a weighted graph, each edge is assigned a weight. In practice, the weight can represent, e.g., the cost of constructing that edge, or the distance between the nodes it connects.

A connected component is a subset of nodes in a graph, where a path exists between every pair of nodes [2]. This path may consist of any positive number of edges. A cycle is when there is a path from a node to itself; this path must not use the same edge twice.

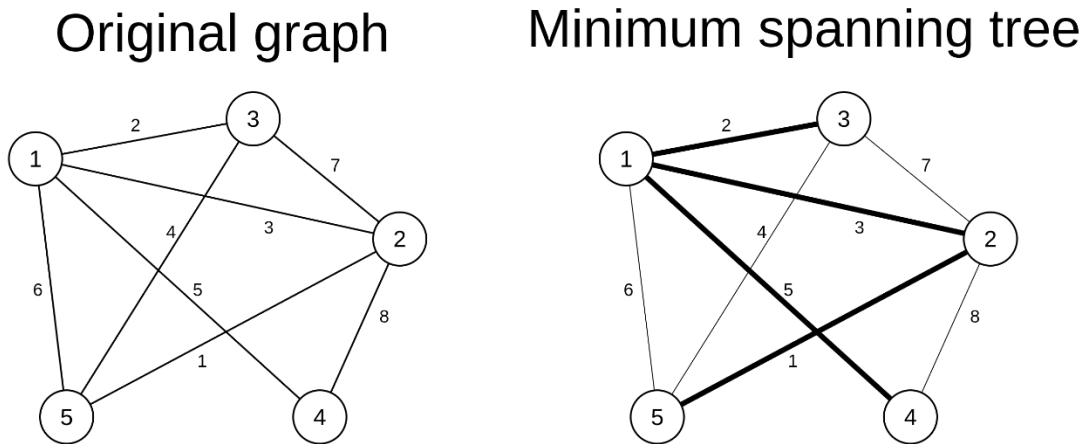
A tree is a type of connected graph without cycles. In a tree with N nodes, it follows that there are exactly $N - 1$ edges [2]. A **spanning tree** is a way of connecting all nodes of a graph with the smallest number of edges possible. An example graph with three possible spanning trees is shown in **Fig. 1**:

Fig 1: Diagram of spanning trees as subgraphs of an original graph G



For a weighted graph, its **minimum** spanning tree is the spanning tree with the smallest sum of edge weights.

Fig. 2: Visualization of a graph with its MST edges bolded



Minimum spanning trees can be very useful. Suppose a water utilities company needs to provide running water to each house in a neighborhood. Using a graph where nodes represent houses and edges represent connections between houses; the cost of each edge would represent the cost of connecting the houses with water pipes. Finding the minimum spanning tree of that graph would give the water utilities company the cheapest way to connect all houses with water.

Graph density represents how many edges a graph has relative to the maximum number of edges possible in that graph [3]. It is calculated using the equation

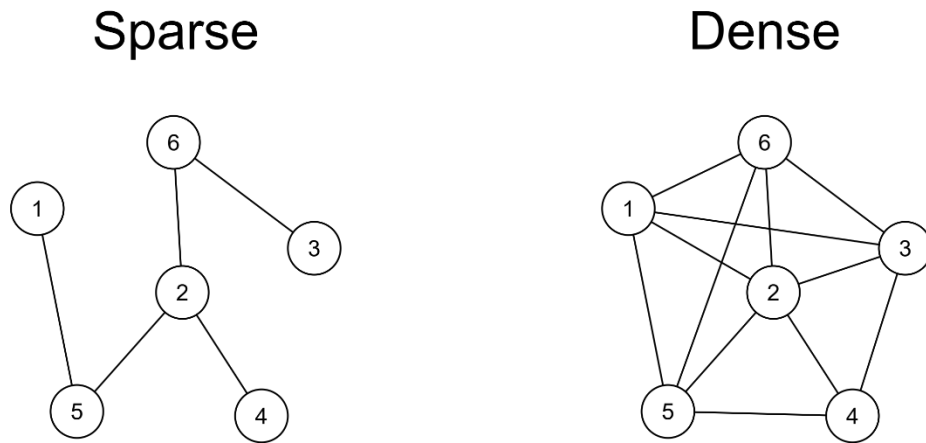
$$D = \frac{\text{number of edges}}{\text{all possible edges}}$$

Where D is the density of the graph.

For a graph with N nodes and E edges, an edge can be constructed between each possible pair of nodes. There are N choose 2 pairs in N nodes therefore the density of G , D_G , is given by:

$$D_G = \frac{E}{\frac{N(N-1)}{2}} = \frac{2E}{N(N-1)}$$

Fig. 3: Sparse vs dense graph with the same number of nodes.

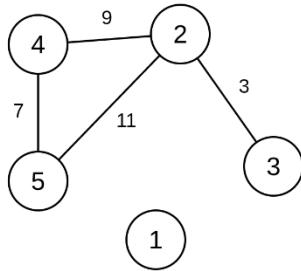


2.1 Graph representation

There are many ways to represent graphs. The two that I will be using to implement Kruskal's and Prim's algorithms are the **edge list** and the **adjacency list**.

Fig. 4: Adjacency list and edge list representations of the same graph

Graph



Adjacency List

Vertices	Adjacencies
1	
2	{3, 3}, {9, 4}, {11, 5}
3	{3, 2}
4	{9, 2}, {7, 5}
5	{11, 2}, {7, 4}

Edge List

{2, 4, 9}
{2, 3, 3}
{2, 5, 11}
{4, 5, 7}

In an **edge list**, all edges are stored in an array of tuples of the form $\{a, b, w\}$ where nodes a and b are connected by an edge with weight w . This allows the algorithm to iterate through and sort all graph edges, which is particularly useful when implementing Kruskal's algorithm.

An **adjacency list** stores, for each node, a list of its neighbors. For a weighted graph, each neighbor is represented as $\{W, V\}$ where W is the weight of the edge to the neighboring node V . This allows efficient traversal of the graph via the neighbors of each node, which is advantageous for Prim's algorithm.

3. Kruskal's Algorithm

Kruskal's algorithm uses the edge list representation to build the MST one edge at a time [4]. First, the edge list is sorted by edge weight, then the algorithm iterates through all edges starting from the smallest. For each edge, the algorithm checks whether including this edge will create a cycle. If the edge will not create a cycle, the edge is added to the tree, otherwise the edge is discarded. After all edges have been processed, for a graph with N nodes, the algorithm will have added $N-1$ edges, and the minimum spanning tree is built.

3.1 Disjoint sets data structure

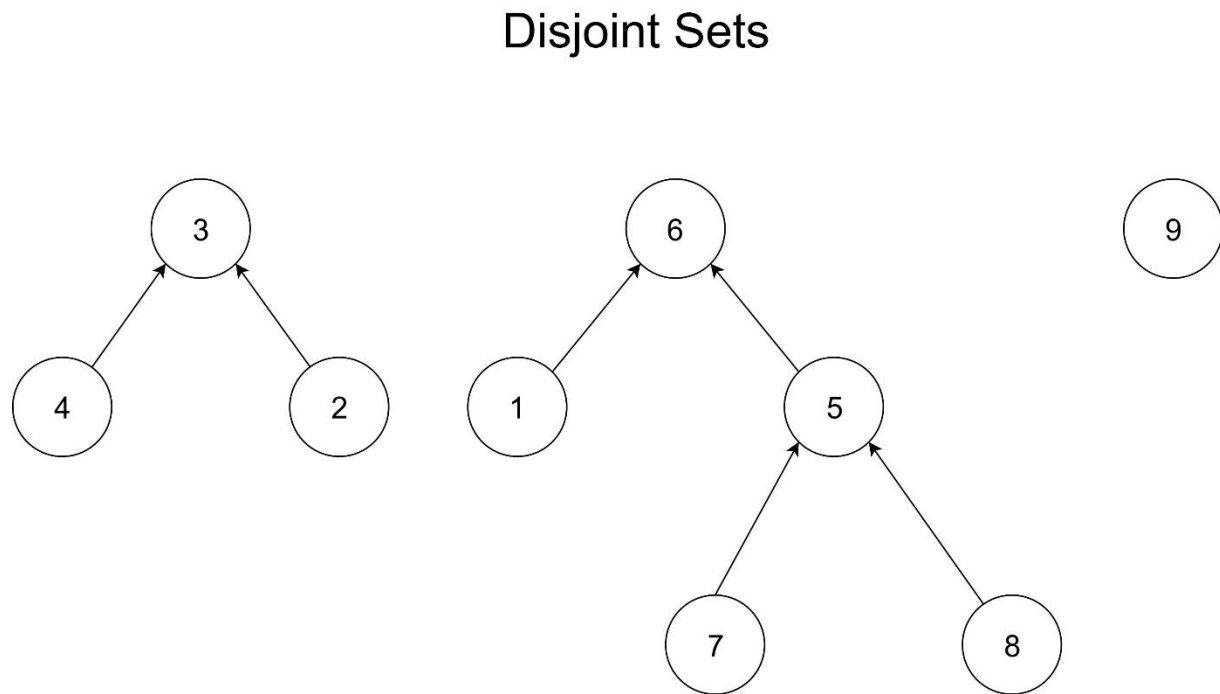
The algorithm requires a way to check whether an edge will create a cycle, which happens when an edge connects two nodes which are already connected. Thus, it requires an efficient data-structure that can check whether two nodes belong to the same component (are already connected).

For this, the disjoint set union (DSU) is used. This data structure organizes the nodes of a graph into sets (components), which are identified by a representative node [5].

- FIND(x) - return the representative node of x 's component
- SAME(x, y) - return whether nodes x and y are in the same component
- UNITE(x, y) - combine the components that contain nodes x and y into one larger component

The following figure shows a visual representation of this data structure.

Fig. 5: Disjoint sets visualization



In Fig. 5 the nodes are in three separate components (which are trees): $\{4, 3, 2\}$, $\{1, 6, 5, 7, 8\}$, $\{9\}$. The representatives of each component are 3, 6 and 9. The FIND function recursively finds the root of a node by following the arrows upward. For example, $\text{FIND}(7)$ returns 6, because we would traverse the nodes $7 \rightarrow 5 \rightarrow 6$. If $\text{SAME}(2, 7)$ were called, the data-structure would first find the representative nodes of 2 and 7, which are 3 and 6 respectively, which are not the same, so the function returns *false*. Say we needed to combine the components of nodes 9 and 2, we would call $\text{UNITE}(9, 2)$, which would first find the representative nodes, which are 9 and 3 respectively. It would then create a connection between these, so that a new bigger component is formed with a single representative node (either 9 or 3).

An optimization that reduces the number of operations in this data-structure is small-to-large merging [5] (or union by size). The FIND function traverses the component one layer at a time, so the maximum number of recursive operations is the depth (the number of layers) of the tree. By

always merging the smaller component to the larger one, to create a new layer, the merged component will need to be at least twice as large as before. In a set of N nodes, it follows that the depth of the component is at most $\log_2(N)$. This means that time complexity of the FIND function is *logarithmic* ($O(\log N)$).

Another optimization to reduce the amount of recursive FIND calls is known as path compression [5]. The idea of path compression is to make the FIND function not only return the root node, but to also make every node along the path point directly to it. This doesn't make the FIND function any faster, but over time it flattens the trees, making each node only one recursive call away from the root node, greatly reducing the amount of future FIND calls.

After these two optimizations, it has been shown that the time complexity of our disjoint sets data structure improves from $O(\log N)$ to $O(\alpha(N))$ ¹, where α is the inverse Ackermann function [6]. The α function grows extremely slowly², which makes the time complexity effectively constant for all practical N .

When implementing the DSU, two arrays are used for space efficiency. One array is to store the representative node of each node of the graph, where each index represents the node and the value represents the node that the node points to. The other array is to store the size of the component represented by each node (like a linked list). Before we have added any edges to a graph, each node is its own root and the size of each component is 1. The pseudocode below is an example implementation of DSU for N nodes, labeled 0 to $N-1$. The vectors LINK and SIZE are of size N .

```
// initialize PARENT and SIZE arrays
loop i=0 TO N-1
```

¹ This is actually the average time complexity over many operations, not worst case.

² It has been shown that the inverse Ackermann function is at most 4 in any realistic computational context ($N < \sim 2^{65536}$) <https://web.archive.org/web/20100120134707/http://kosara.net/thoughts/ackermann42.html>

```

        PARENT[i]=i
        SIZE[i]=1
    end loop

function Find(x)
    //returns the representative (root) node of set containing x
    if PARENT[x]!=x
        PARENT[x]=Find(PARENT[x]) // Path compression
    end if
    return PARENT[x]
end function

//returns whether nodes a and b belong to the same set
function Same(a, b)
    return Find(a) = Find(b)
end function

function Union(a, b)
    //merges nodes a and b into the same set
    ROOT_A = Find(a)
    ROOT_B = Find(b)
    if ROOT_A = ROOT_B
        RETURN // a and b are already in the same set
    end if
    if SIZE[ROOT_A] < SIZE[ROOT_B]
        // small to large optimization
        Swap(ROOT_A, ROOT_B)
    end if
    // merge the two sets
    PARENT[ROOT_B] = ROOT_A
    SIZE[ROOT_A] = SIZE[ROOT_A] + SIZE[ROOT_B]
end function

```

It should be noted that this pseudocode and my C++ implementation later in this investigation are both slightly modified versions of the implementation found in Antti Laaksonen's *Competitive Programmer's Handbook*, for which I have received written permission to use (refer to **Appendix C**).

3.2 Implementing Kruskal's algorithm

To implement Kruskal's algorithm, first sort all edges in non-decreasing order by weight. Then iterate through edges, for each edge if the nodes it connects are not in the same component, add the weight to a variable that stores the total weight of the MST, otherwise discard the edge.

```
TOTALWEIGHT = 0
sort edges of Graph by non-decreasing edge weight
initialize DSU for all nodes in Graph

loop through each edge (A, B, W) in edges
    if not Same(A, B)
        // add edge to MST
        Unite(A, B)
        TOTALWEIGHT = TOTALWEIGHT + W
    end if
end loop

return TOTALWEIGHT
```

To analyze the time complexity of Kruskal's algorithm, the number of operations performed by each part of the algorithm should be considered. Firstly, the sorting algorithm used in this investigation³ sorts all edges, E , in $O(E \log E)$ [7]. Then the algorithm iterates through each edge, and for each edge some amount of FIND calls are made. This runs in time $O(E\alpha(E))$, which is effectively $O(E)$. Total time complexity of Kruskal's algorithm is therefore $O(E \log E)$.

³ std::sort() from C++ Standard Template Library

4. Prim's Algorithm

Prim's algorithm builds the MST starting from an arbitrary node. First, the starting node's edges are added to a list. Then the edge with the smallest weight in the list is chosen and added to the tree. Then, all edges of the newly connected node is added to the list and this process is repeated until all nodes are connected, completing the MST. [4]

Efficient implementation of this algorithm requires a list-like data-structure that always keeps track the smallest element it contains. This can be done efficiently with a min-heap.

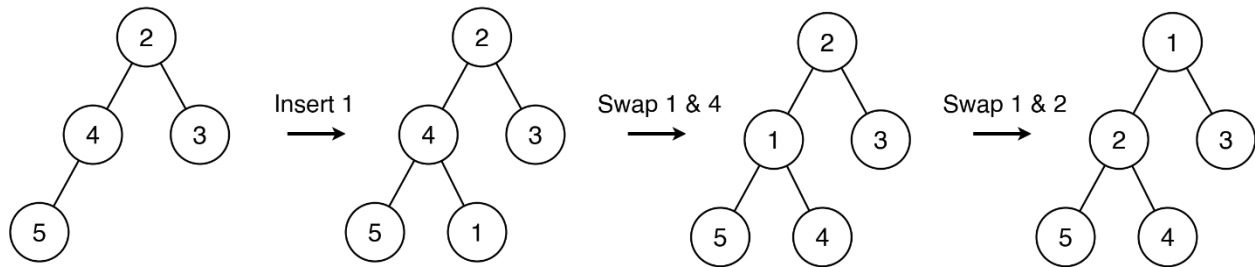
4.1 Min-heap data structure

A min-heap supports three functions: return the smallest element; insert a new element into the queue; and remove the smallest element. In a min-heap, data is organized in a complete binary tree structure, which means every tree level except the bottom is filled. *The tree nodes must satisfy the min-heap property that the parent nodes must always be less than or equal to its children.* This makes it so that the smallest element in the tree is always the root, which means finding the smallest element runs in constant time. [8]

When inserting integers into the min-heap, it first gets inserted as a leaf node. If the parent of this new node is larger than it, they are swapped. This process continues until the parent is smaller or equal to its children.

Fig. 6: Visualization of the min-heap insertion process

Min Heap Insertion

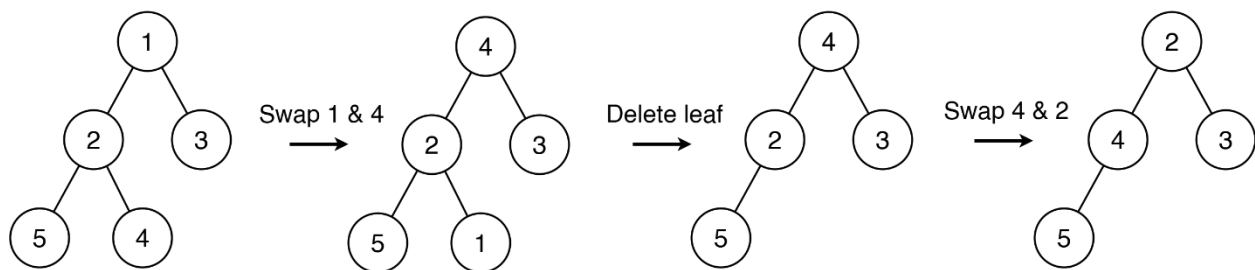


The time complexity of insertion is $O(\log N)$, where N is the number of elements, because the inserted node can be swapped at most $\log_2(N)$ times (the depth of the binary tree).

In a min-heap, the standard deletion operation deletes the minimum element from the root of the tree. This is done by first swapping the root with a leaf node [9]. The leaf is then deleted in $O(1)$ time. This new root node is then swapped downwards until the min-heap property is satisfied. The root node is also swapped down at most $\log_2 N$ layers of the tree, making the time complexity of deletion also $O(\log N)$.

Fig. 7: Visualization of the min-heap deletion process

Min Heap Deletion



It should be noted that for my experiment, I will use the C++ Standard Template Library's `priority_queue` as my min-heap. By default, the STL implementation of priority queue behaves as a max-heap [10]. For this experiment, a custom comparator was used to make it behave like a min-heap. This choice ensures efficient performance and avoids unnecessary complexity in re-implementing a well-tested data structure.

4.2 Implementing Prim's algorithm

When implementing Prim's algorithm, in addition to an adjacency list, a distance array and priority queue are needed. The distance array keeps track of the minimum distance to each node from our current tree. Initially, the tree contains no edges, and one arbitrary node (node 0). The distance to node 0 in the distance array is 0, since the tree contains it, and the distance to all other nodes is infinite. At each node, it adds optimal edges to the priority queue (the edge is considered optimal if the weight of the edge is smaller than the current smallest distance to that node). It then gets the next optimal edge in priority queue and deletes it, repeating this process with the new node of that edge. In the end, once all nodes are added, the algorithm returns the weight of the MST. The pseudocode for prim's algorithm is shown below:

```
TOTALWEIGHT = 0
NODECOUNT = 1
Create min-priority queue PQ
Create array DISTANCE[0..V-1]
// initialize DISTANCE array
loop from i=0 TO V-1
    DISTANCE[i] =  $\infty$ 
end loop

// start building MST from node 0 and distance of 0
Insert (0, 0) into PQ
DISTANCE[0] = 0

while PQ is not empty
    // get smallest edge and the node it connects to from PQ
```

```

    Get minimum edge (W, U) from PQ
    Delete the minimum edge from PQ
    // check if this edge is still optimal to this node
    if DISTANCE[U] != W
        // discard old edge
        continue
    end if
    TOTALWEIGHT = TOTALWEIGHT + W
    DISTANCE[U] = 0
    NODECOUNT = NODECOUNT + 1
    if NODECOUNT = V
        // MST is complete
        return TOTALWEIGHT
    end if
    loop through each (W, V) in Graph[u]
        // check for better edges in adjacency list of v
        if W < DISTANCE[V]
            PUSH(W, V) into PQ
        end if
    end for
end while

```

Suppose the graph in question has V vertices and E edges. The min-heap can hold one edge per vertex not yet in the MST, making the size of the heap V . The algorithm traverses the graph nodes, and at each node the minimum element is accessed in $O(1)$ time, and deleted from the min-heap in $O(\log V)$ time, which makes the time complexity of this part $O(V \log V)$. Each edge is also examined once through the adjacency list, and each time a better edge is found, it is pushed to the heap. In the worst case, this happens to each edge, making the complexity of this part $O(E \log V)$. Therefore, the overall time complexity of Prim's algorithm is $O(E \log V)$. Since V is fixed ($V=2000$) in this investigation, $\log V$ is constant, so the measured growth with respect to E is expected to be close to linear.

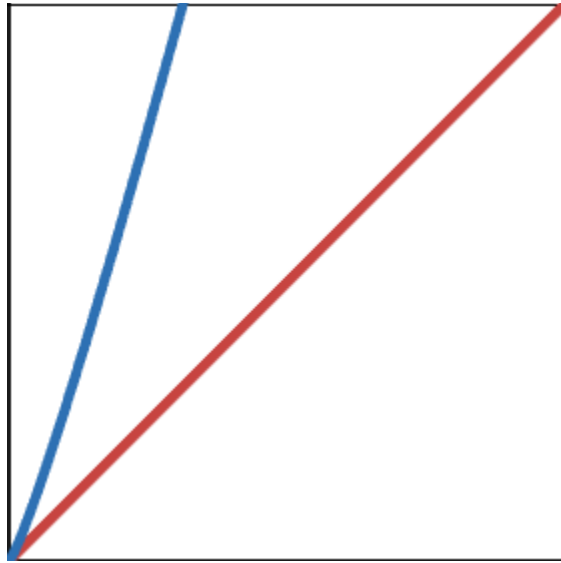
5. Hypothesis and Applied Theory

This experiment will measure the relationship between **time** and **number of edges** in the graph, while keeping the number of nodes constant. Time will be graphed on the **y** axis, and number of edges, E , on the **x** axis. By varying the number of edges on a graph with a constant number of nodes, we can control the density and establish a clear relationship between the runtimes of Kruskal's and Prim's algorithms on graphs of varying densities.

Now that the theory of both algorithms has been covered, it is time to compare the algorithms. Because the number of nodes stays constant, the $\log V$ term in Prim's time complexity is effectively constant. This makes the time complexities of Kruskal's versus Prim's algorithms $O(E \log E)$ versus $O(E)$.

I hypothesize that Prim's algorithm will have a smaller runtime than Kruskal's algorithm, with the difference between them getting larger as the number of edges, E , increases. Kruskal's algorithm is dominated by sorting, giving time complexity $O(E \log E)$. Prim's algorithm with a binary heap is $O(E \log V)$. In this investigation, V is fixed at 2000, so $\log V$ stays constant, and Prim's runtime is expected to scale approximately linearly ($O(E)$) in practice. Therefore, I expect Prim's algorithm to increasingly outperform Kruskal's as E increases. Upon graphing the runtime of Prim's and Kruskal's algorithms, the trend should look like that seen in Fig. 8 below.

Fig. 8: A graph of Kruskal's $O(E \log E)$ (blue) and Prim's $O(E)$ (red) where the x axis is E



6. Methodology

6.1 Independent variable

The independent variable in this experiment is graph density, which will be controlled by manipulating the number of edges, while the number of nodes stays constant at 2000. I will use small incremental values for low densities: 0.0015 ($E \sim 2998$), 0.0020 ($E \sim 3998$), ... 0.01 ($E \sim 19990$), and larger, more spaced out, values for higher densities: 0.02 ($E \sim 39980$), 0.04 ($E \sim 79960$), 0.08 ($E \sim 159920$), 0.16 ($E \sim 319840$). This was chosen because many real-world graphs are sparse, with $E \sim 1-10V$, so I used finer sampling at low densities that represent this range of E [11]. Coarser sampling at higher densities, 0.01-0.16, were done to show long term trends. The highest density sampled was 0.16, which was chosen because preliminary trials demonstrated that its edge count, $E \sim 319840$, was still able to be run in a reasonable amount of time. Each trial graph is generated randomly by a program to maintain consistency while controlling density; this graph is given to both algorithms to allow fair comparison across realistic densities.

6.2 Dependent variable

The dependent variable in this experiment is the time it takes for both algorithms to produce the minimum spanning tree measured in **nanoseconds**. To measure this, I will use the C++ Standard Template Library `high_resolution_clock()` function. Timing started immediately before running Prim's/Kruskal's algorithms and ended immediately after they finished building the MST.

6.3 Controlled variables

Table 1: Table of controlled variables

Variable	Description	Specifications
Number of graph nodes	All graphs in my experiment have a fixed number of nodes (2000) so that I can manipulate graph density by only changing edge count.	
Hardware used	I will be running the program on my home computer.	Operating System: Microsoft Windows Version: Windows 11 24H2 Processor: 3.40 GHz Intel Core i7-14700K RAM: 64GB 6000 MT/s DDR4
Software used	I will be running the program using a text editor	Text Editor: Graphical Vim (gVim) Compiler: g++ (GNU C++ compiler) Version: g++ 14.2.0 Compilation command: g++ -std=c++17 vim/A.cpp -o vim/A -Wl,--stack,268435456
Data type used	All integer operations performed by MST algorithms will be using the 32-bit integer data type	

6.4 Procedure

The procedure followed by my program for my experiment will be as follows:

1. Generate a sample weighted graph with a minimum spanning tree met. The pseudocode for my random graph generation model is below:

```
function generateRandomGraph(V, DENSITY, MAXWEIGHT=1000000)

    create empty graph G
    create empty set EXISTINGEDGES

    MAXEDGES = V * (V - 1) / 2
    TARGETEDGES = DENSITY * MAXEDGES

    // ensure graph is connected by forming a random spanning tree

    create list NODES containing values 0 to V - 1
    randomly shuffle NODES
    // add an edge to each adjacent pair of nodes in NODES
    loop from I = 1 to V - 1
```

```

        U = NODES[I - 1]
        V = NODES[I]
        W = random integer between 1 and MAXWEIGHT

        add edge (U, V) with weight W to G
        add (minimum(U, V), maximum(U, V)) to EXISTINGEDGES
    end loop

    // add additional edges randomly until required density is
    reached

    while size of EXISTINGEDGES < TARGETEDGES
        U = random integer between 0 and V - 1
        V = random integer between 0 and V - 1

        if U ≠ V then
            A = minimum(U, V)
            B = maximum(U, V)

            if (A, B) not in EXISTINGEDGES then
                W = random integer between 1 and MAXWEIGHT

                add edge (A, B) with weight W to G
                add (A, B) to EXISTINGEDGES
            end if
        end if
    end while

    return G
end function

```

2. Input the graph generated by the program in **step 1** into Prim's algorithm and record the runtime of the algorithm in nanoseconds (**Appendix A**)
3. Input the same graph into Kruskal's algorithm and record the runtime in nanoseconds (**Appendix A**)
4. Sanity check the validity of the minimum spanning trees produced by making sure the total weights of the spanning trees created by the Prim() and Kruskal() functions are equal
5. Repeat **steps 2-4** for a total of 20 trials for each density
6. Compute the averages of the runtimes for both algorithms at each density
7. Output the averages into a CSV file

7. Processing data

Table 2: Table showing average runtimes of Kruskal's and Prim's algorithms at each measured density

Density	No. of edges	Average Runtime (nanoseconds)	
		Kruskal's alg.	Prim's alg.
0.0015	2998	1118140	1430710
0.0020	3998	1505820	1809180
0.0025	4997	1921270	2001170
0.0030	5997	2388050	2192740
0.0035	6996	2751110	2307140
0.0040	7996	3237340	2436080
0.0045	8995	3615550	2503110
0.0050	9995	4054870	2564510
0.0055	10994	4426090	2546890
0.0060	11994	5034820	2668760
0.0065	12993	5414220	2762950
0.0070	13993	5795500	2730540
0.0075	14992	6376090	2806580
0.0080	15992	6914450	2841620
0.0085	16991	7247040	2921970
0.0090	17991	7686670	2925790
0.0095	18990	8156180	2897650
0.0100	19990	8624460	3003150
0.0200	39980	18350300	3509130
0.0400	79960	39443800	3967680
0.0800	159920	83961900	4883490
0.1600	319840	174017000	6367910

Fig. 9: Graph of runtime of the MST algorithms (y) against number of edges in the graph (x) for all densities. Key: **RED**: Kruskal's alg. **BLUE**: Prim's alg.

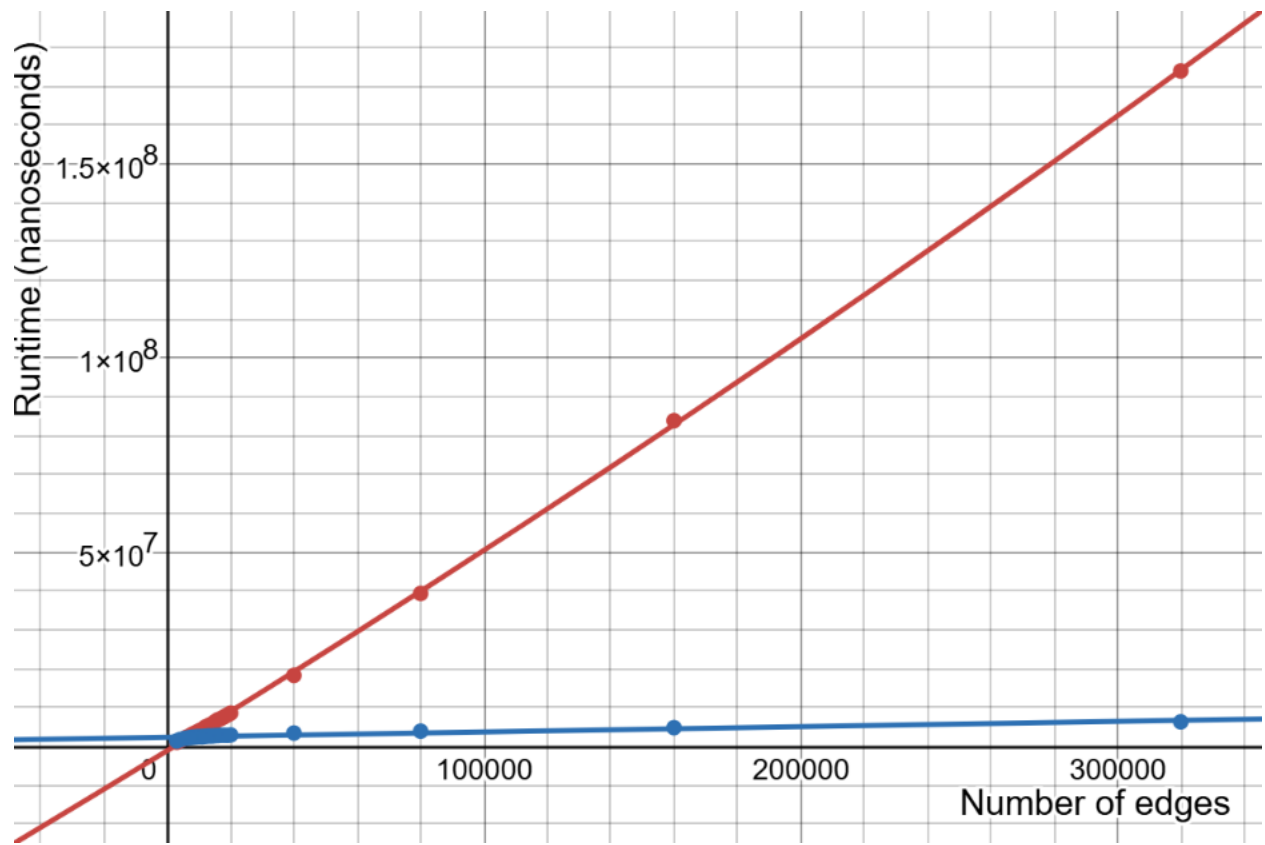
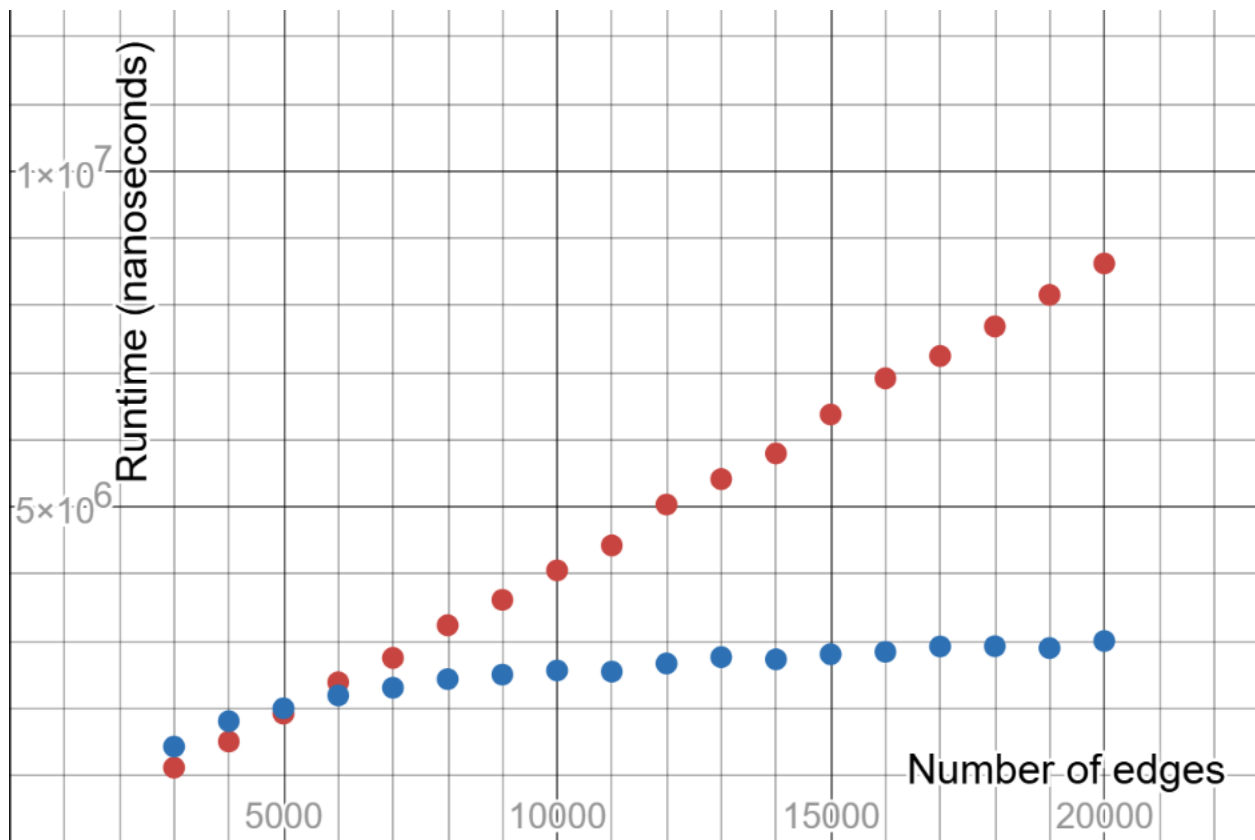


Fig. 10: Graph of runtime of the MST algorithms (y) against number of edges in the graph (x) for only low densities. Key: **RED**: Kruskal's alg. **BLUE**: Prim's alg.



8. Results evaluation and discussion

8.1 Summary of results

Shown in Table 2 and Fig. 10, Kruskal's algorithm is faster at densities 0.0015-0.0025 ($E \sim 2998$, 3998, 4997). From density 0.003 onward ($E \sim 5997$), Prim's algorithm becomes consistently faster, and the gap grows rapidly, for example at $E \sim 319\,840$, Kruskal is $\sim 174\text{ms}$ vs Prim $\sim 6.37\text{ms}$.

8.2 Interpretation and evaluation of results

The general trend seen in Fig. 9 supports my hypothesis: Kruskal's runtime grows much faster than Prim's, like Fig. 8.

Big O predicts the growth rate as E becomes large, but for small E (such as the densities before the crossover point at density ~ 0.003) where the difference in runtime is small, constant factors and details related to my specific implementation can dominate

The Prim variant that I used in my methodology, commonly referred to as the "lazy" version, pushed candidate edges *all* into the min-heap, and old ones are discarded one by one from the heap (instead of directly modifying the value *inside* the heap). This implementation choice could result in the size of the heap exceeding V , sometimes approaching E , which potentially increases the complexity of the insert/delete operations. Additionally, because old edges are discarded very frequently, it increases the constant factors (operations are done many times, which is not acknowledged by Big O). The impacts of this would be most visible in sparse graphs, where runtimes of Kruskal's and Prim's would otherwise be very close, which helps explain why Prim's algorithm runs slower than Kruskal's algorithm at densities 0.0015-0.0025.

8.3 Further evaluation of limitations

The graph generation method I used can slightly bias the results of my experiment in favor of Prim's algorithm. To ensure the graph was connected, I first generated a straight-line like connected structure, before filling in the rest of the edges until the density requirement was met. Prim's algorithm may traverse along this structure without needing to backtrack like in a random MST, improving its empirical runtime. However, I ran preliminary trials with a different graph generation model, and the results were highly similar, so the impact of this is likely to be limited.

The data used in this investigation consists only of averages, but I did not store the individual results used to calculate the averages. Consequently, my results did not show the spread of the runtimes, and I didn't perform any standard deviation calculations, or calculate uncertainty bars for my data, which is a limitation.

9. Conclusion

For connected graphs with ($V=2000$) generated by my random model, Prim's algorithm (using binary heap) scales substantially better than Kruskal's algorithm and is consistently faster at graph densities exceeding ~ 0.003 ($E \sim 6000$).

This matches theoretical expectations because Kruskal's most expensive part is sorting E edges ($O(E \log E)$), whereas Prim's min-heap approach is $O(E \log V)$, which is effectively $O(E)$, with $\log V$ constant in this investigation. However, at very low densities (0.0015 - 0.0025), Kruskal's algorithm ran faster, indicating that for small E (when $E \sim 1.5$ - $2.5V$), constant factors of repeated priority-queue operations can outweigh the extra $\log E$ term of Kruskal's algorithm from sorting.

These conclusions are specific to the implementation choices I made: Prim’s algorithm with duplicate node entries in the min-heap; Kruskal’s using DSU with union by size and path compression optimizations; and my graph generation model. Further explorations could strengthen generality and validity of results by:

1. Calculating spread/error bars for data points.
2. Testing alternative implementations for Prim’s algorithm, for example one that allows direct modification of elements inside the priority queue to limit the size of the heap to V . This could perhaps use a Self-Balancing Binary Search Tree instead of a min-heap.
3. Fixing the number of edges (E) and manipulating the number of nodes (V) to control density instead.
4. Developing alternative graph generation models.

Bibliography

- [1] “Applications of minimum spanning trees.” Accessed: Feb. 21, 2026. [Online]. Available: <https://personal.utdallas.edu/~besp/teaching/mst-applications.pdf>
- [2] “Trees: Chapter 11.9 – 11.10,” *MIT OpenCourseWare*, 2015. https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/mit6_042js15_session21.pdf (accessed Feb. 21, 2026).
- [3] G. D. Luca, “Baeldung,” *Baeldung on Computer Science*, Aug. 24, 2020. <https://www.baeldung.com/cs/graphs-sparse-vs-dense> (accessed Feb. 21, 2026).
- [4] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, Third. Cambridge, Mass.: Mit Press, 2009.
- [5] A. Laaksonen, *Competitive Programmer’s Handbook*. 2018, pp. 145–147. Accessed: Feb. 21, 2026. [Online]. Available: <https://cses.fi/book/book.pdf>

- [6] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, Apr. 1975, doi: <https://doi.org/10.1145/321879.321884>.
- [7] “std::sort - cppreference.com,” *Cppreference.com*, 2025.
<https://en.cppreference.com/w/cpp/algorithm/sort.html> (accessed Feb. 21, 2026).
- [8] GeeksforGeeks, “Introduction to MinHeap – Data Structure and Algorithm Tutorials,” *GeeksforGeeks*, Mar. 22, 2023. <https://www.geeksforgeeks.org/dsa/introduction-to-min-heap-data-structure/> (accessed Feb. 21, 2026).
- [9] GeeksforGeeks, “Insertion and Deletion in Heaps,” *GeeksforGeeks*, Apr. 16, 2019.
<https://www.geeksforgeeks.org/dsa/insertion-and-deletion-in-heaps/> (accessed Feb. 21, 2026).
- [10] “std::priority_queue - cppreference.com,” *Cppreference.com*, 2025.
https://en.cppreference.com/w/cpp/container/priority_queue.html (accessed Feb. 21, 2026).
- [11] Guy Melançon. “Just how dense are dense graphs in the real world? A methodological note.” BELIV 2006: BEyond time and errors: novel evaLuation methods for Information Visualization (AVI Workshop), May 2006, Venice, Italy, pp.75-81. fflirmm-00091354ff

Appendices

Appendix A: Algorithm implementations

A1: Disjoint Set Union (Laaksonen)

```
struct DSU {
    vector<int> link, size;

    DSU(int n) {
        link.resize(n);
        size.assign(n, 1);
        for(int i=0; i<n; i++)
            link[i]=i;
    }

    int find(int x) {
        while(link[x]!=x) {
            link[x] = link[link[x]];
            x = link[x];
        }
        return x;
    }

    bool same(int a, int b) {
        return find(a) == find(b);
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if(a==b) return;
        if(size[a]<size[b]) swap(a, b);
        size[a]+=size[b];
        link[b]=a;
    }
};
```

A2: Kruskal's algorithm

```
int Kruskal(vector<ar<int, 3>>& edges) {
    sort(edges.begin(), edges.end());
    int totalWeight = 0;
    int count = 0;
```

```

DSU dsu(V);
for(auto& [weight, a, b]: edges) {
    if(dsu.same(a, b)) continue;
    dsu.unite(a, b);
    totalWeight += weight;
    count++;
    if(count==V-1) return totalWeight;
}
return totalWeight;
}

```

A3: Prim's algorithm

```

int Prim(const vector<vector<ar<int, 2>>>& adjacencyList) {
    vector<int> d(V, INF);

    priority_queue<ar<int, 2>, vector<ar<int, 2>>, greater<ar<int, 2>>> pq;
    pq.push({d[0]=0, 0});
    int totalWeight=0;
    int count=0;
    while(pq.size()) {
        int node = pq.top()[1];
        int weight = pq.top()[0];
        pq.pop();
        if(d[node]!=weight) continue; // move on if this is old edge
        totalWeight += weight;
        d[node] = 0;
        count++;
        if(count == V) return totalWeight;
        for(auto& [w, node2]:adjacencyList[node]) {
            if(w<d[node2]) {
                pq.push({d[node2]=w, node2});
            }
        }
    }
    return totalWeight;
}

```

Appendix B: Full code used for my experiment

```
#include<bits/stdc++.h>
using namespace std;

#define ar array

const int V = 2000, INF = 1e9;

struct Graph {
    vector<ar<int, 3>> edges;
    vector<vector<ar<int, 2>>> adj;
};

struct DSU {
    vector<int> link, size;

    DSU(int n) {
        link.resize(n);
        size.assign(n, 1);
        for(int i=0; i<n; i++)
            link[i]=i;
    }

    int find(int x) {
        while(link[x]!=x) {
            link[x] = link[link[x]];
            x = link[x];
        }
        return x;
    }

    bool same(int a, int b) {
        return find(a) == find(b);
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if(a==b) return;
        if(size[a]<size[b]) swap(a, b);
        size[a]+=size[b];
        link[b]=a;
    }
};
```

```

    }
};

int Kruskal(vector<ar<int, 3>>& edges) {
    sort(edges.begin(), edges.end());
    int totalWeight = 0;
    int count = 0;
    DSU dsu(V);
    for(auto& [weight, a, b]: edges) {
        if(dsu.same(a, b)) continue;
        dsu.unite(a, b);
        totalWeight += weight;
        count++;
        if(count==V-1) return totalWeight;
    }
    return totalWeight;
}

int Prim(const vector<vector<ar<int, 2>>>& adjacencyList) {
    vector<int> d(V, INF);

    priority_queue<ar<int, 2>, vector<ar<int, 2>>, greater<ar<int, 2>>> pq;
    pq.push({d[0]=0, 0});
    int totalWeight=0;
    int count=0;
    while(pq.size()) {
        int node = pq.top()[1];
        int weight = pq.top()[0];
        pq.pop();
        if(d[node]!=weight) continue; // move on if this is old edge
        totalWeight += weight;
        d[node] = 0;
        count++;
        if(count == V) return totalWeight;
        for(auto& [w, node2]:adjacencyList[node]) {
            if(w<d[node2]) {
                pq.push({d[node2]=w, node2});
            }
        }
    }
    return totalWeight;
}

```

```

Graph generateRandomGraph(double density, int maxWeight = 100000) {
    Graph g;
    g.adj.resize(V);
    set<pair<int, int>> existingEdges;
    random_device seed; // creates a seed
    mt19937 gen(seed()); // generates deterministic random numbers with higher
    randomness than rand()
    uniform_int_distribution<int> generateWeight(1, maxWeight);
    uniform_int_distribution<int> generateNode(0, V-1);

    int maxEdges = V*(V-1) / 2;
    int targetEdges = (int)(density * maxEdges);

    // generate a random spanning tree first to ensure connectedness
    vector<int> nodes(V);
    iota(nodes.begin(), nodes.end(), 0);
    shuffle(nodes.begin(), nodes.end(), gen);

    for(int i=1; i<V; i++) {
        int u = nodes[i-1];
        int v = nodes[i];
        int w = generateWeight(gen);
        g.edges.push_back({w, u, v});
        g.adj[u].push_back({w, v});
        g.adj[v].push_back({w, u});
        existingEdges.insert({min(u, v), max(u, v)});
    }
    // generate the random edges until density requirement is met
    while((int)existingEdges.size() < targetEdges) {
        int u = generateNode(gen);
        int v = generateNode(gen);
        if(u==v) continue;
        if(u>v) swap(u, v);
        if(existingEdges.count({u, v})) continue;
        int w = generateWeight(gen);
        g.edges.push_back({w, u, v});
        g.adj[u].push_back({w, v});
        g.adj[v].push_back({w, u});
        existingEdges.insert({u, v});
    }

    return g;
}

```

```

int main() {
    vector<double> densities = {0.0015, 0.002, 0.0025, 0.003, 0.0035, 0.004, 0.0045,
0.005, 0.0055, 0.006, 0.0065, 0.007, 0.0075, 0.008, 0.0085, 0.009, 0.0095, 0.01,
0.02, 0.04, 0.08, 0.16};
    int trials = 20;

    ofstream out("results.csv");
    out << "density,kruskal_time,prim_time\n";

    for(double density:densities) {
        double kruskalTotal = 0;
        double primTotal = 0;
        bool sanityCheck = 1;
        int kruskalVal, primVal;
        for(int i=0; i<trials; i++) {
            Graph g = generateRandomGraph(density);

            auto start = chrono::high_resolution_clock::now();
            primVal = Prim(g.adj);
            auto end = chrono::high_resolution_clock::now();
            primTotal += chrono::duration<double>(end-start).count();

            auto start2 = chrono::high_resolution_clock::now();
            kruskalVal = Kruskal(g.edges);
            auto end2 = chrono::high_resolution_clock::now();
            kruskalTotal += chrono::duration<double>(end2-start2).count();

            sanityCheck = kruskalVal == primVal;
        }

        out << density << "," << kruskalTotal/trials << "," << primTotal/trials <<
"\n";
        cout << "Correctness check " << (sanityCheck? "passed\n":"failed\n");
        cout << "Done density " << density << "\n";
    }
}

```

Appendix C: Written permission from Antti Laaksonen to use his DSU implementation

C1: Email I sent asking for permission

Dear Mr. Laaksonen,

My name is [REDACTED]. I am a 3rd year student of [REDACTED] where I am currently completing the IB Diploma Program. As a part of the diploma program, I have to complete a 4000-word research paper. The topic I chose for my research is to investigate the difference in the time complexities and runtimes of Prim's and Kruskal's algorithms for the Minimum Spanning Tree problem. I found your implementation of the Disjoint Set Union in section 15.2 of your Competitive Programmer's Handbook.

I was wondering if it would be okay for me to use this implementation (a slightly modified but highly similar implementation) in my experiment and theoretical discussion, given that I cite them. The final essay will not be distributed or sold online, the only people who might gain access to it are me, my supervisor, certain college admissions officers, and an IB examiner who will mark it in the summer of 2026.

Please let me know if this is okay with you, and thank you for taking the time to read this.

Sincerely,
[REDACTED]

C2: His response

Hi [REDACTED]

Sure, you can use this implementation in your research paper.

Best regards,
Antti